Answers to Exercises of Design Patterns and Principles

Exercise-1

Logger File(Class) :

```
package Exercise1;

public class Logger {

    // 1. Private static instance of itself (hidden from the
outside)
    private static Logger instance;

    // 2. Private constructor (prevents anyone else from sayi
ng 'new Logger()')
    private Logger() {
        System.out.println("Logger initialized.");
    }

    // 3. Public static method to get the instance
    public static synchronized Logger getInstance() {
        if (instance == null) {
            instance = new Logger();
        }
        return instance;
    }

    public void log(String message) {
        System.out.println("[LOG]: " + message);
```

```
    }
}
```

TestLogger File(Class) :

```
package Exercise1;

public class TestLogger {
    public static void main(String[] args) {

        System.out.println("Starting application...");

        // First logger instance
        Logger logger1 = Logger.getInstance();
        logger1.log("First log entry.");

        // Second logger instance
        Logger logger2 = Logger.getInstance();
        logger2.log("Second log entry.");

        // TEST: Let's verify that logger1 and logger2 are th
e EXACT same object in memory
        if (logger1 == logger2) {
            System.out.println("SUCCESS: Both variables point
to the exact same Logger instance!");
        } else {
            System.out.println("FAIL: We have multiple instan
ces.");
        }

    }
}
```

Output:

```
Logger initialized.
[LOG]: First log entry.
[LOG]: Second log entry.
SUCCESS: Both variables point to the exact same Logger instance!
```

Exercise-2

Document Interface :

```java
package FactoryMethodPatternExample;

public interface Document {
    void open();
    void save();
}
```

WordDocument Class :

```java
package FactoryMethodPatternExample;

public class WordDocument implements Document {
    @Override
    public void open() {
        System.out.println("Opening Word Document...");
    }

    @Override
    public void save() {
        System.out.println("Saving Word Document...");
    }
}
```

PdfDocument Class:

```
package FactoryMethodPatternExample;

public class PdfDocument implements Document {
    @Override
    public void open() {
        System.out.println("Opening PDF Document...");
    }

    @Override
    public void save() {
        System.out.println("Saving PDF Document...");
    }
}
```

ExcelDocument Class:

```
package FactoryMethodPatternExample;

public class ExcelDocument implements Document {
    @Override
    public void open() {
        System.out.println("Opening Excel Spreadsheet...");
    }

    @Override
    public void save() {
        System.out.println("Saving Excel Spreadsheet...");
    }
}
```

DocumentFactory Class:

```
package FactoryMethodPatternExample;
```

```
public abstract class DocumentFactory {
    public abstract Document createDocument();

    public void manageDocument() {
        Document doc = createDocument();
        doc.open();
        doc.save();
    }
}
```

WordFactory Class:

```
package FactoryMethodPatternExample;

public class WordFactory extends DocumentFactory {
    @Override
    public Document createDocument() {
        return new WordDocument();
    }
}
```

PdfFactory Class:

```
package FactoryMethodPatternExample;

public class PdfFactory extends DocumentFactory {
    @Override
    public Document createDocument() {
        return new PdfDocument();
    }
}
```

ExcelFactory Class:

```
package FactoryMethodPatternExample;

public class ExcelFactory extends DocumentFactory {
    @Override
    public Document createDocument() {
        return new ExcelDocument();
    }
}
```

Main Class:

```
package FactoryMethodPatternExample;

public class Main {
    public static void main(String[] args) {

        System.out.println(" Processing of a 'WORD' Document:
");
        DocumentFactory wordFactory = new WordFactory();
        // We use the factory to manage the document without
ever typing 'new WordDocument()' here
        wordFactory.manageDocument();

        System.out.println(" Processing 'PDF' Document: ");
        DocumentFactory pdfFactory = new PdfFactory();
        pdfFactory.manageDocument();

        System.out.println(" Processing 'EXCEL' Documen: ");
        DocumentFactory excelFactory = new ExcelFactory();
        excelFactory.manageDocument();
    }
}
```

Output :

```
 Processing of a 'WORD' Document:
Opening Word Document...
Saving Word Document...
 Processing 'PDF' Document:
Opening PDF Document...
Saving PDF Document...
 Processing 'EXCEL' Documen:
Opening Excel Spreadsheet...
Saving Excel Spreadsheet...
```

Exercise-3

Computer Class :

```java
package BuilderPatternExample;

public class Computer {

    private String CPU;
    private String RAM;

    private String storage;
    private boolean isGraphicsCardEnabled;
    private boolean isBluetoothEnabled;

    private Computer(ComputerBuilder builder) {
        this.CPU = builder.CPU;
        this.RAM = builder.RAM;
```

```
        this.storage = builder.storage;
        this.isGraphicsCardEnabled = builder.isGraphicsCardEn
abled;
        this.isBluetoothEnabled = builder.isBluetoothEnabled;
    }

    public String getCPU() { return CPU; }
    public String getRAM() { return RAM; }
    public String getStorage() { return storage; }
    public boolean isGraphicsCardEnabled() { return isGraphic
sCardEnabled; }
    public boolean isBluetoothEnabled() { return isBluetoothE
nabled; }

    @Override
    public String toString() {
        return "Computer [CPU=" + CPU + ", RAM=" + RAM + ", S
torage=" + storage +
                ", GPU=" + isGraphicsCardEnabled + ", Bluetoo
th=" + isBluetoothEnabled + "]";
    }

    public static class ComputerBuilder {
        private String CPU;
        private String RAM;
        private String storage;
        private boolean isGraphicsCardEnabled;
        private boolean isBluetoothEnabled;

        // The Builder constructor only takes the REQUIRED pa
rameters
        public ComputerBuilder(String CPU, String RAM) {
            this.CPU = CPU;
            this.RAM = RAM;
        }
```

```
        public ComputerBuilder setStorage(String storage) {
            this.storage = storage;
            return this;
        }

        public ComputerBuilder setGraphicsCardEnabled(boolean
isGraphicsCardEnabled) {
            this.isGraphicsCardEnabled = isGraphicsCardEnable
d;
            return this;
        }

        public ComputerBuilder setBluetoothEnabled(boolean is
BluetoothEnabled) {
            this.isBluetoothEnabled = isBluetoothEnabled;
            return this;
        }

        // 3. The final build() method
        public Computer build() {
            return new Computer(this);
        }
    }
}
```

Main Class :

```
package BuilderPatternExample;

public class Main {
    public static void main(String[] args) {

        System.out.println(" Building a Basic Office Compute
r");
        Computer officePC = new Computer.ComputerBuilder("Int
```

```
el", "8GB")
                .build();
        System.out.println(officePC);


        System.out.println(" Building Optional features");
        Computer gamingPC = new Computer.ComputerBuilder("AM
D", "18GB")
                .setStorage("2TB NVDIA")
                .setGraphicsCardEnabled(true)
                .setBluetoothEnabled(true)
                .build();
        System.out.println(gamingPC);
    }
}
```

Output:

```
 Building a Basic Office Computer
Computer [CPU=Intel, RAM=8GB, Storage=null, GPU=false, Bluetooth=false]
 Building Optional features
Computer [CPU=AMD, RAM=18GB, Storage=2TB NVDIA, GPU=true, Bluetooth=true]
```

Exercise-4

PaymentProcessor Interface :

```
package AdapterPatternExample;

public interface PaymentProcessor {
    void processPayment(double amount);
}
```

StripeGateway Class :

```
package AdapterPatternExample;

public class StripeGateway {
    public void makeCharge(double amountInCents) {
        System.out.println("Stripe: Charged " + amountInCents
+ " cents.");
    }
}
```

PayPalGateway Class :

```
package AdapterPatternExample;


public class PayPalGateway {
    public void sendMoney(String userEmail, double amount) {
        System.out.println("PayPal: Sent $" + amount + " via
account " + userEmail);
    }
}
```

StripeAdapter Class :

```
package AdapterPatternExample;

public class StripeAdapter implements PaymentProcessor {
    private StripeGateway stripeGateway;

    public StripeAdapter(StripeGateway stripeGateway) {
        this.stripeGateway = stripeGateway;
    }

    @Override
    public void processPayment(double amount) {
```

```
        double amountInCents = amount * 100;
        stripeGateway.makeCharge(amountInCents);
    }
}
```

PayPalAdapter Class :

```
package AdapterPatternExample;


public class PayPalAdapter implements PaymentProcessor {
    private PayPalGateway payPalGateway;
    private String userEmail;

    public PayPalAdapter(PayPalGateway payPalGateway, String
userEmail) {
        this.payPalGateway = payPalGateway;
        this.userEmail = userEmail;
    }

    @Override
    public void processPayment(double amount) {
        payPalGateway.sendMoney(userEmail, amount);
    }
}
```

Main Class :

```
package AdapterPatternExample;


public class Main {
    public static void main(String[] args) {

        System.out.println(" Initiating Checkout : ");
```

```
        double orderTotal = 50.00;

        // 1. Using Stripe
        StripeGateway stripeAPI = new StripeGateway();
        PaymentProcessor stripeProcessor = new StripeAdapter
(stripeAPI);

        stripeProcessor.processPayment(orderTotal);


        System.out.println(" Switching Payment Methods : ");

        // 2. Using PayPal
        PayPalGateway payPalAPI = new PayPalGateway();
        PaymentProcessor payPalProcessor = new PayPalAdapter
(payPalAPI, "customer@example.com");

        payPalProcessor.processPayment(orderTotal);
    }
}
```

Output :

```
 Initiating Checkout :
Stripe: Charged 5000.0 cents.
 Switching Payment Methods :
PayPal: Sent $50.0 via account customer@example.com
```

Exercise-5

Notifier Interface :

```
package DecoratorPatternExample;
```

```java
public interface Notifier {
    void send(String message);
}
```

EmailNotifier Class :

```java
package DecoratorPatternExample;

public class EmailNotifier implements Notifier {
    @Override
    public void send(String message) {
        System.out.println("Sending Email: " + message);
    }
}
```

NotifierDecorator Class :

```java
package DecoratorPatternExample;


public abstract class NotifierDecorator implements Notifier {
    private Notifier wrappedNotifier;

    public NotifierDecorator(Notifier notifier) {
        this.wrappedNotifier = notifier;
    }

    @Override
    public void send(String message) {
        wrappedNotifier.send(message);
    }
}
```

SMSNotifierDecorator Class :

```
package DecoratorPatternExample;


public class SMSNotifierDecorator extends NotifierDecorator {

    public SMSNotifierDecorator(Notifier notifier) {
        super(notifier);
    }

    @Override
    public void send(String message) {
        super.send(message);
        System.out.println("Sending SMS: " + message);
    }
}
```

SlackNotifierDecorator Class :

```
package DecoratorPatternExample;

public class SlackNotifierDecorator extends NotifierDecorator
{

    public SlackNotifierDecorator(Notifier notifier) {
        super(notifier);
    }

    @Override
    public void send(String message) {
        super.send(message);
        System.out.println("Sending Slack message: " + messag
e);
    }
}
```

Main Class :

```
package DecoratorPatternExample;

public class Main {
    public static void main(String[] args) {

        System.out.println(" Basic Notification : ");
        Notifier simpleNotifier = new EmailNotifier();
        simpleNotifier.send("Hello World");

        System.out.println(" Layered Notification : ");
        // We wrap the Email base inside an SMS, and wrap tha
t inside a Slack notifier.
        Notifier urgentNotifier = new SlackNotifierDecorator(
                new SMSNotifierDecorator(
                        new EmailNotifier()));

        urgentNotifier.send("CRITICAL SYSTEM FAILURE!");
    }
}
```

Output :

```
 Basic Notification :
Sending Email: Hello World
 Layered Notification :
Sending Email: CRITICAL SYSTEM FAILURE!
Sending SMS: CRITICAL SYSTEM FAILURE!
Sending Slack message: CRITICAL SYSTEM FAILURE!
```

Exercise-6

Image Interface :

```
package ProxyPatterExample;

public interface Image {
    void display();
}
```

RealImage Class :

```
package ProxyPatterExample;

public class RealImage implements Image {
    private String filename;

    public RealImage(String filename) {
        this.filename = filename;
        loadFromRemoteServer();
    }

    private void loadFromRemoteServer() {
        System.out.println("Downloading " + filename + " from
remote server...");
    }

    @Override
    public void display() {
        System.out.println("Displaying :" + filename + " ");
    }
}
```

ProxyImage Class :

```
package ProxyPatterExample;

public class ProxyImage implements Image {
    private RealImage realImage;
    private String filename;

    public ProxyImage(String filename) {
        this.filename = filename;
    }

    @Override
    public void display() {
        if (realImage == null) {
            realImage = new RealImage(filename);
        }
        realImage.display();
    }
}
```

Main Class :

```
package ProxyPatterExample;

public class Main {
    public static void main(String[] args) {

        System.out.println(" System Initialized: ");
        Image image = new ProxyImage("holiday_photo_1080p.jp
g");

        System.out.println(" User clicks 'View Image' for the
first time: ");
        image.display();
```

```
        System.out.println(" User clicks 'View Image' again l
ater: ");
        image.display();
    }
}
```

Output :

```
 System Initialized:
 User clicks 'View Image' for the first time:
Downloading holiday_photo_1080p.jpg from remote server...
Displaying :holiday_photo_1080p.jpg
 User clicks 'View Image' again later:
Displaying :holiday_photo_1080p.jpg
```

Exercise-7

Observer Interface :

```
package ObserverPatterExample;

public interface Observer {
    void update(String stockSymbol, double stockPrice);
}
```

Stock Interface :

```
package ObserverPatterExample;

public interface Stock {
    void registerObserver(Observer observer);
    void deregisterObserver(Observer observer);
```

```
    void notifyObservers();
}
```

StockMarket Class :

```
package ObserverPatterExample;

import java.util.ArrayList;
import java.util.List;

public class StockMarket implements Stock {
    private List<Observer> observers;
    private String stockSymbol;
    private double stockPrice;

    public StockMarket(String stockSymbol) {
        this.observers = new ArrayList<>();
        this.stockSymbol = stockSymbol;
    }

    public void setStockPrice(double stockPrice) {
        this.stockPrice = stockPrice;
        notifyObservers();
    }

    @Override
    public void registerObserver(Observer observer) {
        observers.add(observer);
    }

    @Override
    public void deregisterObserver(Observer observer) {
        observers.remove(observer);
    }
```

```
    @Override
    public void notifyObservers() {
        for (Observer observer : observers) {
            observer.update(stockSymbol, stockPrice);
        }
    }
}
```

MobileAppClient Class:

```
package ObserverPatterExample;

public class MobileAppClient implements Observer {
    private String username;

    public MobileAppClient(String username) {
        this.username = username;
    }

    @Override
    public void update(String stockSymbol, double stockPrice)
{
        System.out.println("Mobile Push to " + username + ":
" + stockSymbol + " is now $" + stockPrice);
    }
}
```

WebAppDashboard Class :

```
package ObserverPatterExample;

public class WebAppDashboard implements Observer {
    @Override
    public void update(String stockSymbol, double stockPrice)
{
```

```
        System.out.println("Web Dashboard Updated: [" + stock
Symbol + "] tick: $" + stockPrice);
    }
}
```

Main Class :

```
package ObserverPatterExample;

public class Main {
    public static void main(String[] args) {

        StockMarket appleStock = new StockMarket("AAPL");

        Observer mobileUser = new MobileAppClient("Veera");
        Observer webDashboard = new WebAppDashboard();

        appleStock.registerObserver(mobileUser);
        appleStock.registerObserver(webDashboard);

        System.out.println(" First Price Update: ");
        appleStock.setStockPrice(150.00);

        System.out.println(" Mobile User Unsubscribes: ");
        appleStock.deregisterObserver(mobileUser);

        System.out.println(" Second Price Update: ");
        appleStock.setStockPrice(155.50);
    }
}
```

Output :

```
 First Price Update:
Mobile Push to Veera: AAPL is now $150.0
Web Dashboard Updated: [AAPL] tick: $150.0
 Mobile User Unsubscribes:
 Second Price Update:
Web Dashboard Updated: [AAPL] tick: $155.5
```

Exercise-8

PaymentStrategy Interface :

```java
package StrategyPatternExample;

public interface PaymentStrategy {
    void pay(int amount);
}
```

CreditCardPayment Class :

```java
package StrategyPatternExample;

public class CreditCardPayment implements PaymentStrategy {
    private String nameOnCard;
    private String cardNumber;

    public CreditCardPayment(String nameOnCard, String cardNu
mber) {
        this.nameOnCard = nameOnCard;
        this.cardNumber = cardNumber;
    }
```

```
    @Override
    public void pay(int amount) {
        System.out.println("Paid $" + amount + " using Credit
Card ending in " + cardNumber.substring(7));
    }
}
```

PayPalPayment Class :

```
package StrategyPatternExample;

public class PayPalPayment implements PaymentStrategy {
    private String email;

    public PayPalPayment(String email) {
        this.email = email;
    }

    @Override
    public void pay(int amount) {
        System.out.println("Paid $" + amount + " using PayPal
account: " + email);
    }
}
```

PaymentContext Class :

```
package StrategyPatternExample;

public class PaymentContext {
    private PaymentStrategy strategy;

    public void setPaymentStrategy(PaymentStrategy strategy)
{
```

```
        this.strategy = strategy;
    }

    public void executePayment(int amount) {
        if (strategy == null) {
            System.out.println("Error: No payment method sele
cted.");
            return;
        }
        strategy.pay(amount);
    }
}
```

Main Class :

```
package StrategyPatternExample;

public class Main {
    public static void main(String[] args) {

        PaymentContext checkout = new PaymentContext();
        int cartTotal = 150;

        System.out.println(" User selects PayPal: ");
        checkout.setPaymentStrategy(new PayPalPayment("veera@
example.com"));
        checkout.executePayment(cartTotal);

        System.out.println(" User changes mind, switches to C
redit Card: ");
        checkout.setPaymentStrategy(new CreditCardPayment("Ve
era Nidhish", "9876543210"));
        checkout.executePayment(cartTotal);
```

```
    }
}
```

Output :

```
 User selects PayPal:
Paid $150 using PayPal account: veera@example.com
 User changes mind, switches to Credit Card:
Paid $150 using Credit Card ending in 210
```

Exercise-9

Command Interface :

```java
package CommandPatternExample;

public interface Command {
    void execute();
}
```

Light Class :

```java
package CommandPatternExample;

public class Light {
    public void turnOn() {
        System.out.println("The Light is ON");
    }

    public void turnOff() {
        System.out.println("The Light is OFF");
```

```
    }
}
```

LightOnCommand Class :

```
package CommandPatternExample;

public class LightOnCommand implements Command {
    private Light light;

    public LightOnCommand(Light light) {
        this.light = light;
    }

    @Override
    public void execute() {
        light.turnOn();
    }
}
```

LightOffCommand Class :

```
package CommandPatternExample;

public class LightOffCommand implements Command {
    private Light light;

    public LightOffCommand(Light light) {
        this.light = light;
    }

    @Override
    public void execute() {
        light.turnOff();
```

```
    }
}
```

RemoteControl Class :

```
package CommandPatternExample;

public class RemoteControl {
    private Command command;

    public void setCommand(Command command) {
        this.command = command;
    }

    public void pressButton() {
        if (command != null) {
            command.execute();
        } else {
            System.out.println("No command assigned to this b
utton.");
        }
    }
}
```

Main Class :

```
package CommandPatternExample;

public class Main {
    public static void main(String[] args) {

        Light livingRoomLight = new Light();

        Command turnOn = new LightOnCommand(livingRoomLight);
        Command turnOff = new LightOffCommand(livingRoomLigh
```

```
t);

        RemoteControl remote = new RemoteControl();

        System.out.println(" Programming and Pressing Button
s: ");

        remote.setCommand(turnOn);
        remote.pressButton();

        remote.setCommand(turnOff);
        remote.pressButton();
    }
}
```

Output :

```
 Programming and Pressing Buttons:
The Light is ON
The Light is OFF
```

Exercise-10

Student Class (Model) :

```
package MVCPatternExample;

public class Main {
    public static void main(String[] args) {

        Student student = new Student("VIT-101", "Veera",
"A");
```

```
        StudentView view = new StudentView();

        StudentController controller = new StudentController
(student, view);

        controller.updateView();

        System.out.println("Updating student grade...");
        System.out.println();
        controller.setStudentGrade("A+");

        controller.updateView();
    }
}
```

StudentView Class (View) :

```
package MVCPatternExample;

public class StudentView {
    public void displayStudentDetails(String studentName, Str
ing studentId, String studentGrade) {
        System.out.println(" Student Record ");
        System.out.println("Name:  " + studentName);
        System.out.println("ID:    " + studentId);
        System.out.println("Grade: " + studentGrade);
        System.out.println();
    }
}
```

StudentController Class (Controller) :

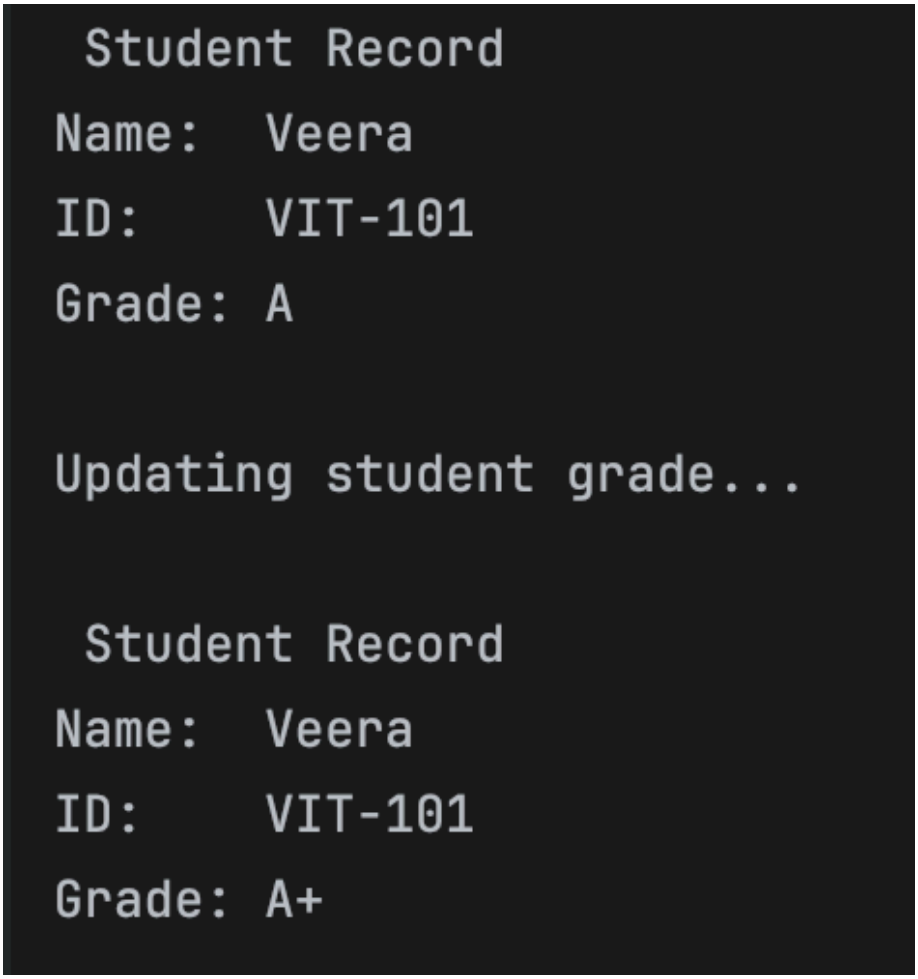
```
package MVCPatternExample;
```

```
public class StudentController {
    private Student model;
    private StudentView view;

    public StudentController(Student model, StudentView view)
{
        this.model = model;
        this.view = view;
    }

    public void setStudentName(String name) {
        model.setName(name);
    }

    public String getStudentName() {
        return model.getName();
    }

    public void setStudentGrade(String grade) {
        model.setGrade(grade);
    }

    public String getStudentGrade() {
        return model.getGrade();
    }

    public void updateView() {
        view.displayStudentDetails(model.getName(), model.get
Id(), model.getGrade());
    }
}
```

Main Class :

```
package MVCPatternExample;

public class Main {
    public static void main(String[] args) {

        Student student = new Student("VIT-101", "Veera",
"A");

        StudentView view = new StudentView();

        StudentController controller = new StudentController
(student, view);

        controller.updateView();

        System.out.println("Updating student grade...");
        System.out.println();
        controller.setStudentGrade("A+");

        controller.updateView();
    }
}
```

Output :



Exercise-11

CustomerRepository Interface :

```
package DependencyInjectionExample;

public interface CustomerRepository {
    String findCustomerById(int id);
}
```

CustomerRepositoryImpl Class :

```
package DependencyInjectionExample;
```

```
public class CustomerRepositoryImpl implements CustomerReposi
tory {
    @Override
    public String findCustomerById(int id) {
        // Simulating a database fetch
        if (id == 101) {
            return "Customer Record: [ID: 101, Name: Veera Ni
dhish]";
        }
        return "Customer not found.";
    }
}
```

CustomerService Class :

```
package DependencyInjectionExample;

public class CustomerService {
    private final CustomerRepository customerRepository;

    public CustomerService(CustomerRepository customerReposit
ory) {
        this.customerRepository = customerRepository;
    }

    public void getCustomerInfo(int id) {
        System.out.println("Processing business logic...");
        String data = customerRepository.findCustomerById(i
d);
        System.out.println(data);
    }
}
```

Main Class :

```
package DependencyInjectionExample;

public class Main {
    public static void main(String[] args) {

        CustomerRepository repository = new CustomerRepositor
yImpl();

        CustomerService service = new CustomerService(reposit
ory);

        System.out.println(" Executing Client Request : ");
        service.getCustomerInfo(101);
    }
}
```

Output :

```
 Executing Client Request :
Processing business logic...
Customer Record: [ID: 101, Name: Veera Nidhish]
```